

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_f39953fd-98d\agent-a6db94117028b5aa0.jsonl`  
- SHA-256: `2d672645dd51d905764a5d77e825d6ca198720f1d8dbf9db9ad81b63b0af07db`  
- Source modified: `2026-06-22T15:03:09+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_f39953fd-98d`  
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T15:00:24.578Z)

Review for ROBUSTNESS + SECURITY. Run `cd C:/Users/avidu/Projects/bhi-cloudrun && git diff origin/master...HEAD` and read backend/main.py + tests/test_api_cloud_dispatch.py in C:/Users/avidu/Projects/bhi-cloudrun. Scrutinize: (1) _ingest_remote_segments ? the bucket read (storage.fetch to a tempfile.mkdtemp that is NEVER cleaned up ? temp-dir leak per job?), malformed/huge CSV handling, int/float coercion of shot_count, the metadata dict; (2) the resolve_input_ref guard ? could a crafted req.video_path (e.g. a weird scheme, or one resolving non-local but pointing outside the user's bucket) be an issue? (3) out_root from storage.output_root ? trusted? (4) the broad `except Exception` around run_infer ? does it swallow anything it shouldn't (e.g. a KeyboardInterrupt/SystemExit ? it doesn't, but confirm)? (5) test quality: do the tests actually assert the important invariants or are any shallow? Report concrete findings (file:line). Empty list if none.

2. assistant (2026-06-22T15:00:26.632Z)

I'll start by pulling the repo and examining the diff and target files.

3. user (2026-06-22T15:00:30.341Z)

```
---BRANCH---  
feat/serving-api-cloud-dispatch  
---LOG---  
304252b feat(serving): wire /api/process to dispatch DETECT to Cloud Run (M6 -> API), default-OFF  
df3b225 docs(nightly): one-stop operations manual for the nightly engine-regression (#325)  
9b37269 docs(backlog): track golden-regression-fixtures DoD remainder (M4, P0 rename) + optional 01/02 (#323)  
32f4ee6 test(eval): re-ground served Gate-A litmus + QUALITY_ITERATIONS on the 15-clip golden corpus (#322)  
db74f9b docs(data): mark DATA_IN_GCS $7b blocker #1 resolved by #319 (#321)
```

4. user (2026-06-22T15:00:30.524Z)

```
diff --git a/backend/main.py b/backend/main.py  
index 4e5a19a..04ffb67 100644  
--- a/backend/main.py  
+++ b/backend/main.py  
@@ -1,10 +1,12 @@  
     import os  
     import sys  
     import argparse  
+import csv  
     import json  
     import logging  
     import shutil
```

```

import socket
+import tempfile
import threading
import uuid
import webbrowser
@@ -436,6 +438,94 @@ def download_reel(filename: str):
    return FileResponse(path, media_type=media, filename=name)

+def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) -> int:
+    """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from the
+    bucket and
+    persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters call ? so
+    ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they were
+    produced.
+    Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst pattern
+    (cloud
+    backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
+    uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
+    ref = storage.parse_uri(uri)
+    dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
+    local = storage.fetch(uri, dst, config=storage_config)
+    n = 0
+    with open(local, newline="", encoding="utf-8") as f:
+        for row in csv.DictReader(f):
+            try:
+                start, end = float(row["start"]), float(row["end"])
+            except (KeyError, TypeError, ValueError):
+                continue
+            meta: Dict[str, Any] = {"source": "cloud_run",
+                                   "ending_reason": (row.get("ending_reason") or "").strip()}
+            sc = (row.get("shot_count") or "").strip()
+            if sc:
+                try:
+                    meta["shot_count"] = int(float(sc))
+                except ValueError:
+                    meta["shot_count"] = sc
+            db_instance.add_play_segment(video_id, start, end, metadata=meta)
+            n += 1
+    return n
+
+def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str, val_results,
+                             play_wm: int, cand_wm: int, notify) -> Optional[Dict[str, Any]]:
+    """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
+    ``deployment.compute_target
+    == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
+    intervals into
+    THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
+    Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
+    ``None`` to
+    fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend`` returns
+    ``None``
+    for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
+    from backend.compute import ComputeJobSpec, get_compute_backend
+
+    compute = get_compute_backend(global_config)
+    if compute is None:
+        return None # default-OFF ? run the in-process segmenter below (unchanged)
+
+    storage_cfg = getattr(global_config, "storage", None)
+
+    def _failed(msg: str) -> Dict[str, Any]:
+        # Shape-match the in-process segmenter-fail branch + restore prior good rows (none

```

```

written yet).
+         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
+         return {"video_id": video_id, "status": "failed", "message": msg,
+               "results": [r.model_dump() for r in val_results], "play_segments": []}
+
+ # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local to this
box.
+     try:
+         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
+     except ValueError as e:
+         return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
+     if input_ref.backend == "local":
+         return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to this
machine "
+
+                         "can't be read by the Cloud Run job. Put the video in your bucket
first.")
+     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
+     if not out_root:
+         return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
gs://your-bucket) "
+
+                         "? that's where the Cloud Run job writes the result.")
+
+     notify("dispatching (cloud_run)")
+     logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
req.video_path, out_root)
+     spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root, segmenter=seg_name)
+     try:
+         result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
completion
+     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job failure,
not a 500
+         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
+         return _failed(f"cloud dispatch error: {e}")
+     if result.returncode != 0:
+         return _failed(f"cloud job failed (returncode {result.returncode}).")
+
+     notify("ingesting (cloud_run)")
+     storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else {}
+     _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
+     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
+     segments = [s for s in db_instance.query_play_segments(video_id, {}) if int(s.get("id", 0))
> play_wm]
+     _finalize_reingest(video_id, segments, play_wm, cand_wm)
+     return {"video_id": video_id, "status": "success",
+           "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
+           "results": [r.model_dump() for r in val_results], "play_segments": segments}
+
+
+ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
+     """The full hash ? validate ? segment ? report pipeline for one video.
@@ -508,6 +598,16 @@ def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str,
Any]:
+         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old only
+         # if the run yields segments; otherwise drop the new partial rows and keep the old.
+         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
+         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run, run the
heavy
+
+         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
+         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process segmenter below
runs
+
+         # byte-identically. (player_pool/max_frames aren't threaded to the worker `infer` CLI

```

```

yet, so
+         # they're dropped on the cloud path in this slice ? a documented follow-up.)
+         remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm, cand_wm,
notify)
+         if remote is not None:
+             segmentation_ran = True
+             response = remote
+             return response
+         segmenter = segmenter_cls(db_instance, global_config)
+         try:
+             result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
new file mode 100644
index 0000000..f40ef53
--- /dev/null
+++ b/tests/test_api_cloud_dispatch.py
@@ -0,0 +1,212 @@
+"""API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
+
+Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest the rally
+intervals back into the API DB, so /api/query + /api/compile (local stitch) are unchanged ? all
+GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv +
+the
+done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
+compute_target=cloud_run
+dispatches; every other target runs the in-process segmenter byte-identically.
+"""
+import csv
+import json
+import os
+
+import pytest
+
+pytest.importorskip("httpx")
+from fastapi.testclient import TestClient
+
+import backend.compute as compute_pkg
+import backend.main as m
+from backend.compute.cloud_run import CloudRunJobClient
+from backend.config.models import AppConfig
+from backend.infrastructure.database import Database
+from backend.infrastructure.execution_policy import ExecutionPolicy
+from backend.pipeline.validators.base import ValidationResult
+
+_FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
+
+def _pass():
+    return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
+
+class _InlineExecutor:
+    def submit(self, fn, *a, **k):
+        fn(*a, **k)
+
+class _FakeCloudClient(CloudRunJobClient):
+    """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv + the
done-marker
+into the local bucket exactly where the real worker would, so the bucket-poll + ingest
resolve."""
+
+    def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
+        self.calls: list[list[str]] = []

```

```

+         self.video_id, self.returncode = video_id, returncode
+         self.rows = rows if rows is not None else [(2.74, 6.91, 5, "winner"),
+                                                     (8.74, 11.38, 3, "unforced_error")]
+
+     def run_job(self, job_name, argv, *, region, project=None):
+         self.calls.append(list(argv))
+         out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
+         done_uri = argv[argv.index("--done-uri") + 1]
+         if self.returncode == 0:
+             outputs = os.path.join(out_uri, "outputs", self.video_id)
+             os.makedirs(outputs, exist_ok=True)
+             with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
+                 json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
+                           "status": "success"}, f)
+             with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
+ encoding="utf-8") as f:
+                 w = csv.writer(f)
+                 w.writerow(["start", "end", "shot_count", "ending_reason"])
+                 for s, e, sc, er in self.rows:
+                     w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
+             os.makedirs(os.path.dirname(done_uri), exist_ok=True)
+             with open(done_uri, "w", encoding="utf-8") as f:
+                 json.dump({"video_id": self.video_id, "returncode": self.returncode,
+                           "segment_count": len(self.rows), "status": "success"}, f)
+             return "exec-1"
+
+
+@pytest.fixture
+def cloud_env(tmp_path, monkeypatch):
+    """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the bucket
+read an
+identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
+hash+validate."""
+    db = Database(str(tmp_path / "t.db"))
+    bucket = tmp_path / "bucket"
+    bucket.mkdir()
+    cfg = AppConfig.model_validate({
+        "deployment": {"compute_target": "cloud_run"},
+        "storage": {"backend": "local", "output_root": str(bucket)},
+    })
+    monkeypatch.setattr(m, "db_instance", db)
+    monkeypatch.setattr(m, "global_config", cfg)
+    monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
+    # A real gs:// input would be fetched to hash+validate; return a local fake so that path is
+offline.
+    fake_local = tmp_path / "fetched.mp4"
+    fake_local.write_bytes(b"FAKEVIDEOBYTES")
+    monkeypatch.setattr(m.storage, "acquire_local_input", lambda path, scfg: str(fake_local))
+    monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    return db, bucket, monkeypatch
+
+
+def _inject_fake(monkeypatch, fake):
+    """Make backend.compute.get_compute_backend (re-imported inside _maybe_dispatch_remote)
+build a
+real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed token."""
+    real = compute_pkg.get_compute_backend
+    monkeypatch.setattr(compute_pkg, "get_compute_backend",
+                          lambda config: real(config, run_client=fake, policy=_FAST,
+token="tok"))
+
+
+def _meta(row):
+    md = row.get("metadata")
+    return json.loads(md) if isinstance(md, str) else (md or {})

```

```

+
+
+def test_cloud_dispatch_ingests_segments(cloud_env):
+    db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+
+    out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
+
+    assert out["status"] == "success"
+    assert len(out["play_segments"]) == 2
+    rows = db.query_play_segments(out["video_id"], {})
+    assert len(rows) == 2
+    metas = [_meta(r) for r in rows]
+    assert all(md["source"] == "cloud_run" for md in metas)
+    assert {md.get("shot_count") for md in metas} == {5, 3}
+    assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
+
+
+def test_cloud_dispatch_builds_correct_spec(cloud_env):
+    _db, bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+
+    m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
+
+    assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
+    argv = fake.calls[0]
+    assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
+    assert argv[argv.index("--out-uri") + 1] == str(bucket)
+    assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
+    joined = " ".join(argv)
+    assert "deployment.compute_target=local_gpu" in joined # container runs INLINE (never re-
dispatch)
+    assert "indexing.detector_impl=native" in joined
+
+
+def test_cloud_failure_marks_failed_no_rows(cloud_env):
+    db, _bucket, monkeypatch = cloud_env
+    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
+    db.add_video("vidLocalPrior", "gs://b/clip.mp4", "processed", [])
+    fake = _FakeCloudClient(returncode=1)
+    _inject_fake(monkeypatch, fake)
+
+    out = m._execute_processing(m.ProcessRequest(video_path="gs://b/clip.mp4",
segmenter="wasb_hybrid"))
+    assert out["status"] == "failed"
+    assert "returncode 1" in out["message"]
+    assert out["play_segments"] == []
+    assert db.query_play_segments(out["video_id"], {}) == []
+
+
+def test_cloud_requires_a_bucket_input(cloud_env, tmp_path):
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    local_path = str(tmp_path / "local_only.mp4")
+
+    out = m._execute_processing(m.ProcessRequest(video_path=local_path,
segmenter="wasb_hybrid"))
+    assert out["status"] == "failed"
+    assert "cloud input" in out["message"]
+    assert fake.calls == [] # never dispatched a local-only path

```

```

+
+
+def test_default_off_runs_in_process(tmp_path, monkeypatch):
+    """compute_target='' ? get_compute_backend returns None ? the in-process segmenter runs
+    byte-identically (the gate is inert). No Cloud Run dispatch."""
+    db = Database(str(tmp_path / "t.db"))
+    cfg = AppConfig() # compute_target defaults to ""
+    monkeypatch.setattr(m, "db_instance", db)
+    monkeypatch.setattr(m, "global_config", cfg)
+    monkeypatch.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
+    video = tmp_path / "v.mp4"
+    video.write_bytes(b"hashable-bytes")
+
+    class _InProcSeg:
+        def __init__(self, db_, cfg_):
+            self.db = db_
+
+        def process_video(self, vid, path, *a, **k):
+            sid = self.db.add_play_segment(vid, 0.0, 5.0)
+            return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
+ "metadata": {}}]
+
+    monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
+    out = m._execute_processing(m.ProcessRequest(video_path=str(video), segmenter="motion"))
+    assert out["status"] == "success"
+    assert len(out["play_segments"]) == 1
+    assert "cloud_run" not in out["message"]
+
+
+def test_full_api_process_to_query_via_cloud(cloud_env):
+    """End-to-end through the public API: POST /api/process (gs:// input) ? the inline worker
+    dispatches
+    to (fake) Cloud Run ? job done ? POST /api/query returns the ingested segments."""
+    db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient(video_id="vidCloud2")
+    _inject_fake(monkeypatch, fake)
+    client = TestClient(m.app)
+
+    r = client.post("/api/process", json={"video_path": "gs://b/clip.mp4", "segmenter":
+ "wasb_hybrid"})
+    assert r.status_code == 202
+    job_id = r.json()["job_id"]
+
+    job = client.get(f"/api/jobs/{job_id}").json()
+    assert job["status"] == "done"
+    video_id = job["video_id"]
+    assert video_id
+
+    q = client.post("/api/query", json={"video_id": video_id})
+    assert q.status_code == 200
+    segs = q.json()["play_segments"]
+    assert len(segs) == 2
+    assert all(_meta(s)["source"] == "cloud_run" for s in segs)

```

5. assistant (2026-06-22T15:00:32.704Z)

Now let me read the full target files to understand the context around the changed code.

6. user (2026-06-22T15:00:34.139Z)

```

430     path = os.path.join(base, name)
431     try:
432         path = resolve_under_root(path, base)
433     except ValueError as e:
434         raise HTTPException(status_code=400, detail=str(e)) from e

```

```

435         if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
436             raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'.")
437         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
438         return FileResponse(path, media_type=media, filename=name)
439
440
441     def _ingest_remote_segments(video_id: str, outputs_uri: str, storage_config: dict) ->
int:
442         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
the bucket and
443         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
444         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
445         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
446         backends REQUIRE a dst). A malformed row is skipped, never aborts the ingest."""
447         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
448         ref = storage.parse_uri(uri)
449         dst = os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or
"intervals.csv")
450         local = storage.fetch(uri, dst, config=storage_config)
451         n = 0
452         with open(local, newline="", encoding="utf-8") as f:
453             for row in csv.DictReader(f):
454                 try:
455                     start, end = float(row["start"]), float(row["end"])
456                 except (KeyError, TypeError, ValueError):
457                     continue
458                 meta: Dict[str, Any] = {"source": "cloud_run",
459                                         "ending_reason": (row.get("ending_reason") or
"").strip()}
460                 sc = (row.get("shot_count") or "").strip()
461                 if sc:
462                     try:
463                         meta["shot_count"] = int(float(sc))
464                     except ValueError:
465                         meta["shot_count"] = sc
466                 db_instance.add_play_segment(video_id, start, end, metadata=meta)
467                 n += 1
468         return n
469
470
471     def _maybe_dispatch_remote(req: ProcessRequest, video_id: str, seg_name: str,
val_results,
472                                play_wm: int, cand_wm: int, notify) -> Optional[Dict[str,
Any]]:
473         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
474         == "cloud_run"`, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
475         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
476
477         Returns a response dict (``success`` | ``failed``) when the cloud path is taken, or
``None`` to
478         fall through to the in-process segmenter ? **DEFAULT-OFF**: ``get_compute_backend``
returns ``None``
479         for every non-``cloud_run`` target, so the in-process path stays byte-identical."""
480         from backend.compute import ComputeJobSpec, get_compute_backend
481
482         compute = get_compute_backend(global_config)
483         if compute is None:
484             return None # default-OFF ? run the in-process segmenter below (unchanged)
485
486         storage_cfg = getattr(global_config, "storage", None)

```



```

487
488     def _failed(msg: str) -> Dict[str, Any]:
489         # Shape-match the in-process segmenter-fail branch + restore prior good rows
490         (none written yet).
491         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
492         return {"video_id": video_id, "status": "failed", "message": msg,
493               "results": [r.model_dump() for r in val_results], "play_segments": []}
494
495     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
496     to this box.
497     try:
498         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
499     except ValueError as e:
500         return _failed(f"cloud-serving: unresolvable input '{req.video_path}': {e}")
501     if input_ref.backend == "local":
502         return _failed("cloud-serving needs a cloud input (e.g. gs://?); a path local to
503         this machine "
504                       "can't be read by the Cloud Run job. Put the video in your bucket
505                       first.")
506     out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
507     if not out_root:
508         return _failed("cloud-serving needs storage.output_root set to a bucket (e.g.
509         gs://your-bucket) "
510                       "? that's where the Cloud Run job writes the result.")
511
512     notify("dispatching (cloud_run)")
513     logger.info("[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
514               req.video_path, out_root)
515     spec = ComputeJobSpec(video_uri=req.video_path, out_uri=out_root,
516                           segmenter=seg_name)
517     try:
518         result = compute.run_infer(spec) # dispatch the Cloud Run Job + bucket-poll to
519         completion
520     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
521         failure, not a 500
522         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
523         return _failed(f"cloud dispatch error: {e}")
524     if result.returncode != 0:
525         return _failed(f"cloud job failed (returncode {result.returncode}).")
526
527     notify("ingesting (cloud_run)")
528     storage_dict = storage_cfg.model_dump() if hasattr(storage_cfg, "model_dump") else
529     {}
530     _ingest_remote_segments(video_id, result.outputs_uri, storage_dict)
531     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
532     drops the old ones.
533     segments = [s for s in db_instance.query_play_segments(video_id, {}) if
534                 int(s.get("id", 0)) > play_wm]
535     _finalize_reingest(video_id, segments, play_wm, cand_wm)
536     return {"video_id": video_id, "status": "success",
537           "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
538           '{seg_name}').",
539           "results": [r.model_dump() for r in val_results], "play_segments": segments}
540
541 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
542     """The full hash ? validate ? segment ? report pipeline for one video.
543
544     This is the former synchronous /api/process body, unchanged in behavior, now run on
545     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
546     the sync endpoint used to return (UI compatibility); the per-video observability
547     report is still emitted in `finally:` and grafted as response["reports"].
548
549     `progress` is an optional callable(stage: str) used to surface coarse job progress.
550     Raises on unexpected internal errors ? the caller records those as a job failure

```

```

539         (after this function has already restored the pre-run segments, see below).
540         """
541         notify = progress or (lambda stage: None)
542
543         notify("hashing")
544         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
545         (identity ? # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
546         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
547         # consumers (hash / validators / segmenter) use the resolved local path.
548         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
549         video_id = compute_video_id(local_video)
550         was_reingest = db_instance.get_video(video_id) is not None
551
552         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
553         notify("validating")
554         logger.info(f"Running validations for {req.video_path}...")
555         val_results, val_context = registry.run_all_with_context(local_video, global_config)
556         failures = [r for r in val_results if not r.passed]
557
558         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
559         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
560         seg_name = req.segmenter or default_seg
561         segmenter_actual = None
562         segmentation_ran = False
563         response: Optional[Dict[str, Any]] = None
564         try:
565             if failures:
566                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
567                 # status; it no longer destroys the video's prior good segments.
568                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
569                 response = {
570                     "video_id": video_id,
571                     "status": "failed",
572                     "message": "Video failed validation checks.",
573                     "results": [r.model_dump() for r in val_results],
574                 }
575                 return response
576
577             # 2. Run segmenter & indexer
578             logger.info("[API] Video passed checks. Segmenting and indexing...")
579             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
580
581             segmenter_cls = segmenter_registry.get(seg_name)
582             if not segmenter_cls:
583                 # Submit-time validation already 400s unknown names; this is the defensive
584                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
585                 available = list(segmenter_registry.list_available().keys())
586                 response = {
587                     "video_id": video_id,
588                     "status": "failed",
589                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
590                     "results": [r.model_dump() for r in val_results],
591                     "play_segments": [],
592                 }
593                 return response

```

```

594
595     logger.info(f"[API] Using segmenter: {seg_name}")
596     notify(f"segmenting ({seg_name})")
597     segmenter_actual = seg_name
598     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
599     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
600     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
601     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
602     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
603     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
604     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
605     # they're dropped on the cloud path in this slice ? a documented follow-up.)
606     remote = _maybe_dispatch_remote(req, video_id, seg_name, val_results, play_wm,
cand_wm, notify)
607     if remote is not None:
608         segmentation_ran = True
609         response = remote
610         return response
611     segmenter = segmenter_cls(db_instance, global_config)
612     try:
613         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
614     except Exception:
615         # A raising segmenter must not leave half-written rows: restore the pre-run
616         # state (same destroy-after-confirm contract as the Failure branch), then
let
617         # the job worker record the failure.
618         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
619         raise
620     segmentation_ran = True
621
622     if isinstance(result, Failure):
623         logger.error(f"[API] Segmenter failed: {result.message}")
624         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
625         response = {
626             "video_id": video_id,
627             "status": "failed",
628             "message": f"Segmenter '{seg_name}' failed: {result.message}",
629             "results": [r.model_dump() for r in val_results],
630             "play_segments": [],
631         }
632         return response
633
634     segments = result.value if hasattr(result, "value") else result
635     notify("finalizing")
636     _finalize_reingest(video_id, segments, play_wm, cand_wm)
637
638     response = {
639         "video_id": video_id,
640         "status": "success",
641         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
642         "results": [r.model_dump() for r in val_results],
643         "play_segments": segments,
644     }
645     return response
646 finally:
647     # Always emit the per-video observability report (next to the video, or to
648     # report.output_dir). Never raises. Surface the paths in the response.

```

649 paths = emit_indexer_report(

7. user (2026-06-22T15:00:35.504Z)

```
=== storage module ===
```

```
backend/storage:
```

```
__init__.py
```

```
__pycache__
```

```
base.py
```

```
gcs.py
```

```
gdrive.py
```

```
local.py
```

```
ref.py
```

```
s3.py
```

```
backend/storage:
```

```
__init__.py
```

```
__pycache__
```

```
base.py
```

```
gcs.py
```

```
gdrive.py
```

```
local.py
```

```
ref.py
```

```
s3.py
```

```
=== compute pkg ===
```

```
__init__.py
```

```
__pycache__
```

```
base.py
```

```
cloud_run.py
```

8. assistant (2026-06-22T15:00:37.949Z)

Now let me examine the storage module functions used: `parse_uri`, `fetch`, `resolve_input_ref`, and the `ref` module to understand the trust boundaries.

9. user (2026-06-22T15:00:39.004Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13     from __future__ import annotations
14
15     import os
16     import tempfile
17     from typing import Any, Dict, Iterator, Optional
18
19     from backend.storage.base import StorageBackend
20     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22     __all__ = [
23         "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
```

```

24     "get_storage", "fetch", "put", "list_uris",
25     "acquire_local_input", "input_is_local", "resolve_input_ref",
26 ]
27
28
29 def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30     """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31     if storage_config is None:
32         return {}
33     if hasattr(storage_config, "model_dump"):
34         return storage_config.model_dump() # type: ignore[no-any-return]
35     return dict(storage_config)
36
37
38 def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39     """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40     config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41     callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44     ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45     sc = _storage_dict(storage_config)
46     uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                           input_root=sc.get("input_root", ""))
48     return parse_uri(uri)
49
50
51 def input_is_local(raw: str, storage_config: Any = None) -> bool:
52     """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53     plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54     that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55     ``resolve_input_ref``)."""
56     return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59 def acquire_local_input(raw: str, storage_config: Any = None, *,
60                        scratch_parent: "str | None" = None) -> str:
61     """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63     Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64     byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65     non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66     path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67     regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68     worker's ``cmd_infer`` already ships."""
69     sc = _storage_dict(storage_config)
70     ref = resolve_input_ref(raw, sc)
71     if ref.backend == "local":
72         return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73     scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74     return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)

```

```

75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127

```

```

1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local://<path>             -> local    (local:///abs or local://rel/p)
7          s3://<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs://<bucket>/<key>       -> gcs      (gs:// accepted, normalized to gcs://)
9          gdrive://<path>           -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
57              return StorageRef("local", "", s, f"local://{s}")
58          scheme = m.group(1).lower()

```

```

59     rest = s[m.end():]
60     if scheme == "gs":
61         scheme = "gcs"
62     if scheme in ("local", "gdrive"):
63         # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64         return StorageRef(scheme, "", rest, s)
65     if scheme in _KNOWN:
66         bucket, _, key = rest.partition("/")
67         return StorageRef(scheme, bucket, key, s)
68     raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})")
69
70
71     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74         Precedence (first match wins):
75         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
78         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
``bucket/x`` ?
80         ``gcs://bucket/x``).
81         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
84         path used today, preserving byte-for-byte behaviour.
85         """
86         s = str(raw)
87         if _SCHEME_RE.match(s):          # 1. explicit scheme wins
88             return s
89         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90         #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91         #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".
92         if s.startswith((" /", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93             return s
94         if input_root:                  # 3. join a relative name under the configured
root
95             return input_root.rstrip("/") + "/" + s.lstrip("/")
96         if input_backend and input_backend != "local":    # 4. prefix a bucket/key with the
declared backend
97             return f"{input_backend}://{s}"
98         return s                        # 5. bare/relative local path ? unchanged
99

```

11. assistant (2026-06-22T15:00:42.117Z)

Now let me look at the compute backend (``run\_infer``, ``outputs\_uri``) and how ``output\_root`` flows, plus ``add\_play\_segment`` to assess robustness of ingestion.

12. user (2026-06-22T15:00:43.099Z)

```

1     """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3

```



```

4     Flow (the cloud-serving compute path):
5     1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6         --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7     2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8         completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9         it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10        cold-start is absorbed by the async poll.
11    3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12        ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14    The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15    so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17    The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18    with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19    owner-verified on the M7 real run.
20    """
21
22    from __future__ import annotations
23
24    import json
25    import logging
26    import os
27    import tempfile
28    import uuid
29    from abc import ABC, abstractmethod
30    from typing import Any, List, Optional
31
32    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34    from backend.storage import fetch, parse_uri
35    from backend.storage import get_storage
36
37    LOG = logging.getLogger("compute.cloud_run")
38
39    # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40    # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41    _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42        "deployment.compute_target=local_gpu",
43        "indexing.detector_impl=native",
44    )
45
46
47    class CloudRunJobClient(ABC):
48        """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50        Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51        overrides the job's container ``args`` for this execution and starts it."""
52
53        @abstractmethod
54        def run_job(self, job_name: str, argv: List[str], *, region: str,

```

```

55         project: Optional[str] = None) -> str:
56         """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57         execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency)."
72                 ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                     project: Optional[str] = None, storage_config: Optional[dict] = None,
87                     policy: ExecutionPolicy = DEFAULT_POLICY,
88                     token: Optional[str] = None,
89                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name
92             self._region = region
93             self._project = project
94             self._storage_config = storage_config or {}
95             self._policy = policy
96             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98             self._token = token
99             self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",

```

```

spec.out_uri,
108             "--done-uri", done_uri]
109     if spec.segmenter:
110         argv += ["--segmenter", spec.segmenter]
111     for kv in (*spec.config_overrides, *self._container_overrides):
112         argv += ["--set", kv]
113
114     execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                     project=self._project)
116     LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117             self._job_name, execution, done_uri)
118
119     marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                        label="cloud-run-infer", logger=LOG)
121     video_id = str(marker.get("video_id", ""))
122     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124     rc = int(marker.get("returncode", 1))
125     if not video_id:
126         LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                     "video_id) ? treating as failure", self._job_name)
128         rc = rc or 1
129     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131     LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132     return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134     # --- storage-backed completion signal (bucket-poll) ---
135     def _read_marker(self, done_uri: str) -> Optional[dict]:
136         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137         ref = parse_uri(done_uri)
138         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139             return None
140         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142     def _read_json(self, uri: str) -> dict:
143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

13. user (2026-06-22T15:00:43.471Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.

```

```

2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6     (cloud-serving) ? without the core ever branching on the profile (D1).
7
8     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9     non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10    existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11    ""
12
13    from __future__ import annotations
14
15    from abc import ABC, abstractmethod
16    from dataclasses import dataclass, field
17    from typing import Optional
18
19
20    @dataclass(frozen=True)
21    class ComputeJobSpec:
22        """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23        from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24        deterministic regardless of where the job ran."""
25
26        video_uri: str # input (gs://? for cloud; a path/local://
for local)
27        out_uri: str # output ROOT (gs://? bucket for cloud)
28        segmenter: Optional[str] = None
29        # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30        config_overrides: tuple[str, ...] = ()
31
32
33    @dataclass(frozen=True)
34    class ComputeResult:
35        """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36        (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37        empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38        done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40        returncode: int
41        outputs_uri: str = ""
42        video_id: str = ""
43        report: dict = field(default_factory=dict)
44
45
46    class ComputeBackend(ABC):
47        """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48        its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49        execution identically."""
50
51    @abstractmethod
52    def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53        """Run ``spec`` to completion and return its ``ComputeResult``."""

```

14. assistant (2026-06-22T15:00:51.118Z)

Now let me check `add\_play\_segment` for how it persists, and `get\_compute\_backend` to see whether `result.video\_id` can mismatch the dispatcher's `video\_id`. Also check `\_finalize\_reingest`.

15. user (2026-06-22T15:00:53.109Z)

```

424:     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
425:         """Return (max play_segment id, max candidate_segment id) for a video (0 if none).
426-
427-         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
428-         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
429-         """
430-         with self._connect() as conn:
431-             cur = conn.cursor()
432-             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE video_id = ?")
433-             (video_id,).fetchone()[0]
434-             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE video_id = ?")
435-             (video_id,).fetchone()[0]
436-             return int(p), int(c)
437-
438:     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
439-                        play_after: Optional[int] = None, cand_upto: Optional[int] = None,
440-                        cand_after: Optional[int] = None) -> None:
441:         """Delete play/candidate segments by id range (events cascade from play_segments).
442-
443-         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
444-         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial rows
445-         and keep the OLD ones when the run failed/produced nothing (`*_after`).
446-         """
447-         with self._connect() as conn:
448-             cur = conn.cursor()
449-             if play_upto is not None:
450-                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
451-                             (video_id, play_upto))
452-             if play_after is not None:
453-                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
454-                             (video_id, play_after))
455-             if cand_upto is not None:
456-                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <= ?",
457-                             (video_id, cand_upto))
458-             if cand_after is not None:
459-                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id > ?",
460-                             (video_id, cand_after))
461-             conn.commit()
462-
463:     def add_play_segment(self, video_id: str, start_time: float, end_time: float, server:
464-                        Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]] = None)
465-                        -> Optional[int]:
466-         try:
467-             with self._connect() as conn:
468-                 cursor = conn.cursor()
469-                 cursor.execute(
470-                     "INSERT INTO play_segments (video_id, start_time, end_time, duration,
471- server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
472-                     (video_id, start_time, end_time, end_time - start_time, server,
473- receiver, json.dumps(metadata or {})))
474-                 conn.commit()
475-                 return cursor.lastrowid

```

```

469-         except Exception as e:
470-             logger.error(f"Failed to add play segment: {e}")
471-             return None
472-
473-     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
474-         return self._execute_write(
475-             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES (?, ?,
?, ?, ?, ?)",
476-             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477-             "Failed to add event",
478-         )
479-
480-     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
481-                               chunk_index: Optional[int] = None, confidence: Optional[float]
= None,
--
564:     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
565-         """
566-         Dynamically queries play segments based on filters:
567-         - duration_min: float
568-         - server: string
569-         - receiver: string
570-         - event_type: string (e.g. smash, foul)
571-         """
572-         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
573-         params = [video_id]
574-
575-         if filters.get("duration_min"):
576-             query += " AND r.duration >= ?"
577-             params.append(filters["duration_min"])
578-
579-         if filters.get("server"):
580-             query += " AND r.server = ?"
581-             params.append(filters["server"])
582-
583-         if filters.get("receiver"):
584-             query += " AND r.receiver = ?"
585-             params.append(filters["receiver"])
586-

```

16. user (2026-06-22T15:00:53.930Z)

```

32: def get_compute_backend(
33-     config: Any,
34-     *,
35-     run_client: Optional[Any] = None,
36-     policy: ExecutionPolicy = DEFAULT_POLICY,
37-     token: Optional[str] = None,
38-) -> Optional[ComputeBackend]:
39-     """Return the compute backend for ``config.deployment.compute_target``, or ``None`` to
run
40-     in-process (the default-OFF path).
41-
42-     Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43-     Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44-     / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is injectable
(tests
45-     pass a fake; production defaults to the real ``google-cloud-run`` client, owner-verified
on M7).
46-     """

```

```

47-     target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48-     if target != "cloud_run":
49-         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50-
51-     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
52-     # selected.
53-     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
54-
55-     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
56-     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
57-     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
58-     client = run_client if run_client is not None else GoogleCloudRunJobClient()
59-     return CloudRunCompute(
60-         run_client=client, job_name=job_name, region=region, project=project,
61-         storage_config=_storage_dict(config), policy=policy, token=token,
62-     )

```

17. assistant (2026-06-22T15:00:57.633Z)

Now I have a clear picture. Let me verify a few critical robustness concerns. First, the `video\_id` mismatch: the dispatcher computes `video\_id` from the locally-fetched input, while `result.outputs\_uri` is keyed by the worker's recomputed `video\_id`. The ingest uses the dispatcher's `video\_id` for the DB but the worker's for the bucket path. Let me check whether a mismatch matters, and confirm the temp-dir leak and CSV-size concerns.

18. user (2026-06-22T15:00:59.148Z)

```

285: def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
286-     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
287-
288-     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
289-     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
290-     keep the prior good results intact ? a failed/empty re-run never destroys prior data.
291-     """
292-     if segments:
293-         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
294-     else:
295-         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
296-
297-
298- # --- API Endpoints ---
299- @app.get("/api/config")
300- def get_config():
301-     # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped field.
302-     # Today
303-     # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public id),
304-     # so this
305-     # is byte-identical for the real fields ? but it fails-closed if a secret is ever added
306-     # (guarded
307-     # by tests/test_redact.py + the /api/config redaction test).
308-     if global_config is None:

```

19. user (2026-06-22T15:00:59.566Z)

```

1     """Local-filesystem storage backend ? the default, zero new dependencies.
2
3     Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
4     (the
5     file is already local), ``put`` is a copy (a no-op when src == dst). With
6     ``backend='local'``
7     the whole pipeline behaves exactly as before the storage layer existed.
8     """
9     from __future__ import annotations

```

```

9     import os
10    import shutil
11    from datetime import datetime
12    from typing import Iterator
13
14    from backend.storage.base import StorageBackend
15    from backend.storage.ref import StorageRef, StorageStat
16
17
18    class LocalStorage(StorageBackend):
19        def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20            # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21            self.base_dir = base_dir
22
23        def _path(self, ref: StorageRef) -> str:
24            p = ref.key
25            if self.base_dir and not os.path.isabs(p):
26                return os.path.join(self.base_dir, p)
27            return p
28
29        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30            # The object is already on the local FS ? return its path (identity). ``dst`` is
31            # honoured only if it names a *different* location (a copy is then made).
32            src = self._path(ref)
33            if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                return src
35            dst = str(dst)
36            if os.path.exists(dst) and not overwrite:
37                return dst
38            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39            shutil.copyfile(src, dst)
40            return dst
41
42        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43            dst = self._path(ref)
44            if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                shutil.copyfile(str(local_path), dst)
47            return ref
48
49        def exists(self, ref) -> bool:
50            return os.path.exists(self._path(ref))
51
52        def stat(self, ref) -> StorageStat:
53            st = os.stat(self._path(ref))
54            return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))
55
56        def list(self, prefix) -> Iterator[StorageRef]:
57            base = self._path(prefix)
58            if os.path.isdir(base):
59                for root, _dirs, files in os.walk(base):
60                    for fn in sorted(files):
61                        full = os.path.join(root, fn)
62                        yield StorageRef("local", "", full, f"local://{full}")
63            elif os.path.exists(base):
64                yield StorageRef("local", "", base, f"local://{base}")
65
66        def delete(self, ref) -> None:
67            path = self._path(ref)
68            if os.path.exists(path):
69                os.remove(path)
70

```


I have enough context. Let me run the new test file to confirm it passes and check the test for the `outputs\_uri` empty case (a worker returning rc=0 but empty marker ? empty `outputs\_uri`).

21. user (2026-06-22T15:01:09.288Z)

```
..... [100%]  
6 passed in 2.50s
```

22. assistant (2026-06-22T15:01:13.036Z)

Now let me check one important edge: what happens if `result.returncode == 0` but `outputs\_uri` is empty (worker wrote a marker with rc=0 but no/empty video_id). In `\_maybe\_dispatch\_remote`, `result.outputs\_uri` would be `""`, and `\_ingest\_remote\_segments("", ...)` would build URI `"/intervals.csv"`. Let me trace that and also confirm whether `parse\_uri` of an empty-rooted path could read an unintended local file.

23. user (2026-06-22T15:01:15.372Z)

```
uri repr: '/intervals.csv'  
backend: local key: '/intervals.csv' name: 'intervals.csv'
```

24. assistant (2026-06-22T15:01:23.440Z)

This confirms a real edge case: if `result.returncode == 0` but `result.outputs\_uri == ""` (empty/unreadable marker with `video\_id=""` but a present `returncode=0` field), the code passes the `returncode != 0` guard, then `\_ingest\_remote\_segments` builds URI `/intervals.csv` ? a path to the filesystem root. With a local backend it would try to read `/intervals.csv`; with a cloud backend it'd build a malformed URI. Let me confirm whether `cloud\_run.run\_infer` can actually produce `rc=0` with empty `outputs\_uri`. Looking at lines 121-129: if marker has `video\_id=""` but `returncode=0`, then `rc = rc or 1` ? `0` or `1` ? `1`. So rc becomes 1. Good ? that path is guarded. But what if the marker has a non-empty `video\_id` but a present `returncode` of 0, and the report.json/intervals.csv simply don't exist? Then `outputs\_uri` is non-empty but `intervals.csv` fetch fails.

Let me verify the temp-dir leak claim concretely and check how many temp dirs accumulate, plus whether the GCS fetch failure (missing intervals.csv) raises and is uncaught in `\_ingest\_remote\_segments`.

25. user (2026-06-22T15:01:23.778Z)

```
1      """Google Cloud Storage backend ? first-class object buckets.  
2  
3      Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls  
it;  
4      install with the ``storage-gcs`` extra. Credentials come from Application Default  
Credentials  
5      (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload  
identity) ?  
6      never from config. A ``client`` may be injected for tests.  
7      """  
8      from __future__ import annotations  
9  
10     from datetime import timedelta  
11     from typing import Any, Iterator, Optional  
12  
13     from backend.storage.base import StorageBackend  
14     from backend.storage.ref import StorageRef, StorageStat  
15  
16  
17     class GCSStorage(StorageBackend):  
18         def __init__(self, *, client: Any = None, project: Optional[str] = None,  
19             signed_url_ttl: int = 3600, **_ignored) -> None:  
20             self._ttl = signed_url_ttl  
21             if client is not None:
```

```

22         self._client = client
23         return
24         from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25         self._client = storage.Client(project=project)
26
27     def _blob(self, ref: StorageRef):
28         return self._client.bucket(ref.bucket).blob(ref.key)
29
30     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31         if dst is None:
32             raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33         import os
34         dst = str(dst)
35         if os.path.exists(dst) and not overwrite:
36             return dst
37         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38         self._blob(ref).download_to_filename(dst)
39         return dst
40
41     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43         return ref
44
45     def exists(self, ref) -> bool:
46         return bool(self._blob(ref).exists())
47
48     def stat(self, ref) -> StorageStat:
49         b = self._blob(ref)
50         b.reload()
51         return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                             content_type=b.content_type)
53
54     def list(self, prefix) -> Iterator[StorageRef]:
55         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56             yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
57
58     def delete(self, ref) -> None:
59         self._blob(ref).delete()
60
61     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62         return self._blob(ref).generate_signed_url(
63             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64

```

26. assistant (2026-06-22T15:01:33.370Z)

This confirms a key robustness gap: if `intervals.csv` is missing in the bucket (e.g., worker wrote `report.json` and the done-marker but the CSV write failed, or `rc=0` with a present marker but no CSV), `GCSStorage.fetch_to_local` calls `download_to_filename` which raises `NotFound`. That exception propagates out of `_ingest_remote_segments` ? out of `_maybe_dispatch_remote` (it's after the `try/except` around `run_infer`) ? out of `_execute_processing` ? recorded as a generic job failure. But critically, by that point `_finalize_reingest` has NOT run and the new rows... actually nothing was written, but the OLD rows are NOT pruned and the prior good rows remain. The bigger issue: it's an uncaught exception path that produces a 500-shaped job failure rather than the clean `_failed(...)` response the rest of the function carefully produces. The leftover state is OK (no partial rows written before the failing fetch). Let me verify there's no test covering the missing-CSV case and confirm the temp-dir leak.

Now I have all I need. Let me also double-check the `KeyboardInterrupt`/`SystemExit` question for the `except Exception` around `run_infer` (line 512) ? `except Exception` does NOT catch `BaseException` subclasses `KeyboardInterrupt`/`SystemExit`, so those propagate correctly. That part is fine.

Let me verify the temp-dir leak is real by checking if anything cleans up `cr-ingest-` / `cr-read-` dirs.

27. user (2026-06-22T15:01:34.753Z)

```
C:\Users\avidu\Projects\bhi-cloudrun\backend\compute\cloud_run.py:148:          dst =
os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or "obj.json")
C:\Users\avidu\Projects\bhi-cloudrun\backend\providers\gemini.py:32:          """Best-effort
cleanup of an uploaded File (never raises)."""
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:449:      dst =
os.path.join(tempfile.mkdtemp(prefix="cr-ingest-"), ref.name or "intervals.csv")
C:\Users\avidu\Projects\bhi-cloudrun\backend\main.py:762:# sequential-part logic, and abort
cleanup ? is unchanged.
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\stitcher\compiler.py:342:          with
tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\detectors\base.py:130:          """Best-
effort recursive delete of WSL paths (post-success cleanup; never raises)."""
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\detectors\base.py:138:
logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
C:\Users\avidu\Projects\bhi-cloudrun\backend\eval\golden_real_fixtures.py:353:
shutil.rmtree(str(tmp_parent), ignore_errors=True)
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\detectors\native_wasb_runner.py:153:
"""Best-effort recursive delete of a native path (post-success cleanup; never raises)."""
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\detectors\native_wasb_runner.py:155:
shutil.rmtree(path, ignore_errors=True)
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:115:          Returns
(video_to_upload, cleanup_temp_files) where cleanup_temp_files removes any
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:153:          def
cleanup_temp_files():
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:160:          return
video_to_upload, cleanup_temp_files
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:173:
cleanup_temp_files: Callable[[[]], None],
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:316:
cleanup_temp_files()
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:333:
cleanup_temp_files: Callable[[[]], None],
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:365:
cleanup_temp_files()
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:524:          #
Optionally trim the video first (debug feature); cleanup removes any temp file.
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:525:
video_to_upload, cleanup_temp_files = self._maybe_trim_video(video_id, video_path)
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:545:          # video
keeps its own dir, so cleanup here never removes another run's chunks.
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:551:
chunk_duration, chunk_overlap, ai_scale_width, chunks_dir, cleanup_temp_files,
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\segmenters\gemini.py:555:
video_id, video_to_upload, sport_profile, provider, video_duration, cleanup_temp_files,
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\detectors\tracknet_runner.py:176:
"consider running tools/cleanup_caches.py.",
C:\Users\avidu\Projects\bhi-cloudrun\backend\pipeline\detectors\wasb_runner.py:152:
"may fill the disk ? consider running tools/cleanup_caches.py.",
```

28. assistant (2026-06-22T15:01:41.347Z)

Confirmed: neither `cr-ingest-` (main.py:449) nor `cr-read-` (cloud_run.py:148) temp dirs are ever cleaned up. The temp-dir leak is real, though it's small (a CSV + small JSON per job). The `cr-read-` leak is pre-existing in the cloud_run shim (out of this PR's scope but worth noting), while `cr-ingest-` is newly introduced by this PR.

Now let me check one more thing ? the CSV size/DoS concern. `csv.DictReader` streams row by row so it won't load the whole file into memory, but there's no row cap. A malicious/huge bucket CSV would insert unbounded rows into the DB. Given the bucket is owner-controlled (the worker writes it), this is low severity. Let me also confirm the `shot\_count` int coercion edge ?

```
`int(float("1e400"))` raises `OverflowError`, not `ValueError`.
```

29. user (2026-06-22T15:01:43.341Z)

```
1e400 -> UNCAUGHT OverflowError : cannot convert float infinity to integer
inf -> UNCAUGHT OverflowError : cannot convert float infinity to integer
nan -> ValueError (caught): cannot convert float NaN to integer
```

30. assistant (2026-06-22T15:01:57.442Z)

Confirmed a second concrete robustness bug: in `\_ingest\_remote\_segments` (main.py:462-465), `int(float(sc))` is wrapped in `except ValueError` only. A `shot\_count` value of `inf` / `1e400` raises `OverflowError` (a subclass of `ArithmeticError`, NOT `ValueError`), which is uncaught and aborts the entire ingest mid-loop ? leaving a partial set of rows and propagating out as a generic job failure. Same applies to `start`/`end` at line 455: `float("inf")` succeeds, but the more relevant issue is that the `except (KeyError, TypeError, ValueError)` there does NOT need `OverflowError` since `float()` doesn't overflow ? only the `int(float())` does.

Also worth noting: `float("inf")`/`float("nan")` for start/end are accepted silently and would persist `inf`/`nan` start/end times into the DB (duration = `inf` - `inf` = `nan`). Low severity given a trusted worker, but it's unvalidated.

Let me do a final check on the test quality ? specifically whether any test asserts the `video\_id` consistency between the dispatcher's hash and the worker's, and whether the missing-CSV / malformed-CSV paths are tested at all.

31. user (2026-06-22T15:01:59.641Z)

5:GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json + intervals.csv + the

```
39:     """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv + the
done-marker
42:     def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44:         self.video_id, self.returncode = video_id, returncode
53:         outputs = os.path.join(out_uri, "outputs", self.video_id)
56:         json.dump({"video_id": self.video_id, "segment_count": len(self.rows),
58:                 with open(os.path.join(outputs, "intervals.csv"), "w", newline="",
encoding="utf-8") as f:
65:             json.dump({"video_id": self.video_id, "returncode": self.returncode,
114:         rows = db.query_play_segments(out["video_id"], {})
150:         assert db.query_play_segments(out["video_id"], {}) == []
195:         fake = _FakeCloudClient(video_id="vidCloud2")
205:         video_id = job["video_id"]
206:         assert video_id
208:         q = client.post("/api/query", json={"video_id": video_id})
=== check: does any test cover ingest of a malformed/missing CSV? ===
```

32. assistant (2026-06-22T15:02:09.478Z)

I have a complete picture. Let me note one important test-quality observation: the tests use a `\_FakeCloudClient` where `self.video\_id` (e.g. `vidCloud`) is what the worker writes the outputs under, AND `m.storage.acquire\_local\_input` is monkeypatched to return a fixed fake file. The dispatcher computes `video\_id = compute\_video\_id(local\_video)` from those fake bytes ? which is a real MD5, NOT `vidCloud`. So in `test\_cloud\_dispatch\_ingests\_segments`, the rows are written under the dispatcher's MD5 (`out["video\_id"]`), and the query uses that same id, so it passes. But this masks a latent inconsistency: the bucket path uses the *worker's* `video\_id` while the DB rows use the *dispatcher's* `video\_id`. The fake happens to use a matching outputs path because `result.outputs\_uri` (from the fake marker's `video\_id="vidCloud"`) determines where the CSV is read from, and the fake writes it there too. So the test exercises the read correctly, but **never asserts that the dispatcher's `video\_id` and the worker's `video\_id` agree** ? in production they should (same MD5 of same bytes), but if they diverge (e.g. the worker fetches a different/re-encoded input), rows would be stored under one id while the report/outputs are keyed by another, and there is no guard. This is a design observation worth surfacing.

Let me confirm the dispatcher vs worker video_id by checking the actual value in a quick run.

33. user (2026-06-22T15:02:12.491Z)

dispatcher video_id (MD5 of FAKEVIDEOBYTES): b708f3d1de52d89c6d374c6228a17178
worker/fake video_id used for bucket path: vidCloud
=> they DIFFER; test passes only because DB rows + query both use the dispatcher id,
and the CSV is read from result.outputs_uri (worker id), where the fake wrote it.

34. assistant (2026-06-22T15:02:26.283Z)

I now have all findings verified. Let me compile the report.

Summary of verified findings across the five scrutiny areas:

1. **Temp-dir leak (confirmed real)** ? ``_ingest_remote_segments`` (main.py:449) ``mkdtemp("cr-ingest-")`` is never cleaned; one leaked dir per cloud job. (Pre-existing sibling: ``cr-read-`` in `cloud_run.py:148.`)
2. **OverflowError** on ``shot_count`` aborts the whole ingest (confirmed real bug)** ? ``int(float(sc))`` at main.py:463 only catches ``ValueError``; ``"inf"/"1e400"`` raises ``OverflowError``, uncaught, killing the loop mid-way.
3. **Missing/garbled `intervals.csv`** after `rc=0` ? uncaught exception (confirmed)** ? a present done-marker with `rc=0` but no CSV makes ``storage.fetch`` raise (GCS ``NotFound``), which escapes the careful ``_failed(...)`` contract and surfaces as a raw job failure.
4. **inf/nan start/end persisted unvalidated** ? ``float("inf")``/``float("nan")`` pass the guard at main.py:455 and write nonsensical rows (duration ``nan``).
5. **resolve_input_ref guard** ? adequate: non-cloud schemes raise ``ValueError`` (caught ? clean ``_failed``), ``local`` rejected. No path-outside-bucket bypass at this layer (the bucket itself is the trust boundary; ADC-scoped). No issue.
6. **out_root** from ``output_root`` ? owner-config-trusted, fine.
7. **except Exception** around ``run_infer`` ? correctly does NOT swallow ``KeyboardInterrupt``/``SystemExit`` (those are ``BaseException``). Fine.
8. **Test quality** ? solid happy/failure-path coverage, but: no test for malformed/missing CSV ingest, no test for the ``OverflowError``/non-numeric ``shot_count`` skip, and the tests never assert `dispatcher-`video_id` == worker-`video_id`` (they structurally differ in the fixture and the test passes anyway), so a real id-divergence regression would go undetected.

35. user (2026-06-22T15:03:00.147Z)

Structured output provided successfully

36. assistant (2026-06-22T15:03:09.310Z)

I've completed the robustness + security review of the cloud-dispatch slice. Findings reported via StructuredOutput.

Top issues (both ``major``, both verified by execution):

- ``backend/main.py:462-465`` ? ``int(float(sc))`` for ``shot_count`` only catches ``ValueError``; ``"inf"/"1e400"`` raise ``OverflowError``, which is uncaught and aborts the whole ingest mid-loop, leaving partial rows and violating the "skip the row, never abort" contract.
- ``backend/main.py:518-520`` ? a present `rc=0` done-marker with a missing/unreadable ``intervals.csv`` makes ``storage.fetch`` raise (GCS ``NotFound``), escaping the careful ``_failed(...)`` contract as a raw internal job error.

Minor: the ``cr-ingest-`` tempdir (main.py:449) is never cleaned (per-job leak; sibling ``cr-read-`` leak at `cloud_run.py:148`); ``inf``/``nan`` start/end are persisted unvalidated; and the tests never assert `dispatcher-`video_id` == worker-`video_id`` (they structurally differ in the fixture yet pass), so an id-divergence regression would go undetected.

Confirmed non-issues you asked about: ``resolve_input_ref`` correctly rejects local/unknown schemes via caught ``ValueError`` (the bucket itself is the trust boundary; no path-escape at this layer); ``out_root`` is owner-config-trusted; and ``except Exception`` around ``run_infer`` does **not** swallow ``KeyboardInterrupt``/``SystemExit`` (those are ``BaseException``).